

SESSION THIRTEEN: ADVANCED DATA VISUALIZATION IN PYTHON

Learning Outcomes

By the end of this session, learners should be able to:

- Customize matplotlib visualizations.
- Create multiple plots using subplots and GridSpec.
- Build interactive visualizations using Plotly and Bokeh.
- Produce statistical graphics using Seaborn.
- Create geospatial maps using GeoPandas and Folium.
- Select appropriate visualization techniques for different data types.

Required Libraries

```
In [1]: #!/pip install folium  
#!/pip install bokeh  
#!/pip install geopandas
```

```
In [2]: import numpy as np  
import pandas as pd  
  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
import plotly.express as px  
from bokeh.plotting import figure, show  
from bokeh.io import output_notebook  
  
import geopandas as gpd  
import folium  
import bokeh
```

Creating Sample Dataset

```
In [3]: np.random.seed(123)  
  
data = pd.DataFrame({  
    "Age": np.random.randint(18,65,100),  
    "Income": np.random.normal(50000,12000,100),  
    "Score": np.random.normal(70,12,100),  
    "Gender": np.random.choice(["Male","Female"],100),  
})
```

```

    "Department": np.random.choice(
        ["IT", "Finance", "HR", "Marketing"],
        100)
    })

data.head()

```

```

Out[3]:

```

	Age	Income	Score	Gender	Department
0	63	51288.242024	73.903923	Female	Marketing
1	20	32246.874330	84.063710	Female	IT
2	46	49886.422200	94.024309	Female	Finance
3	52	52445.943039	65.368698	Male	IT
4	56	50893.343100	88.664286	Male	Finance

Customizing Matplotlib Plots

Matplotlib provides complete control over figure appearance.

Example 1: Customized Scatter Plot

```

In [4]: plt.figure(figsize=(8,6))

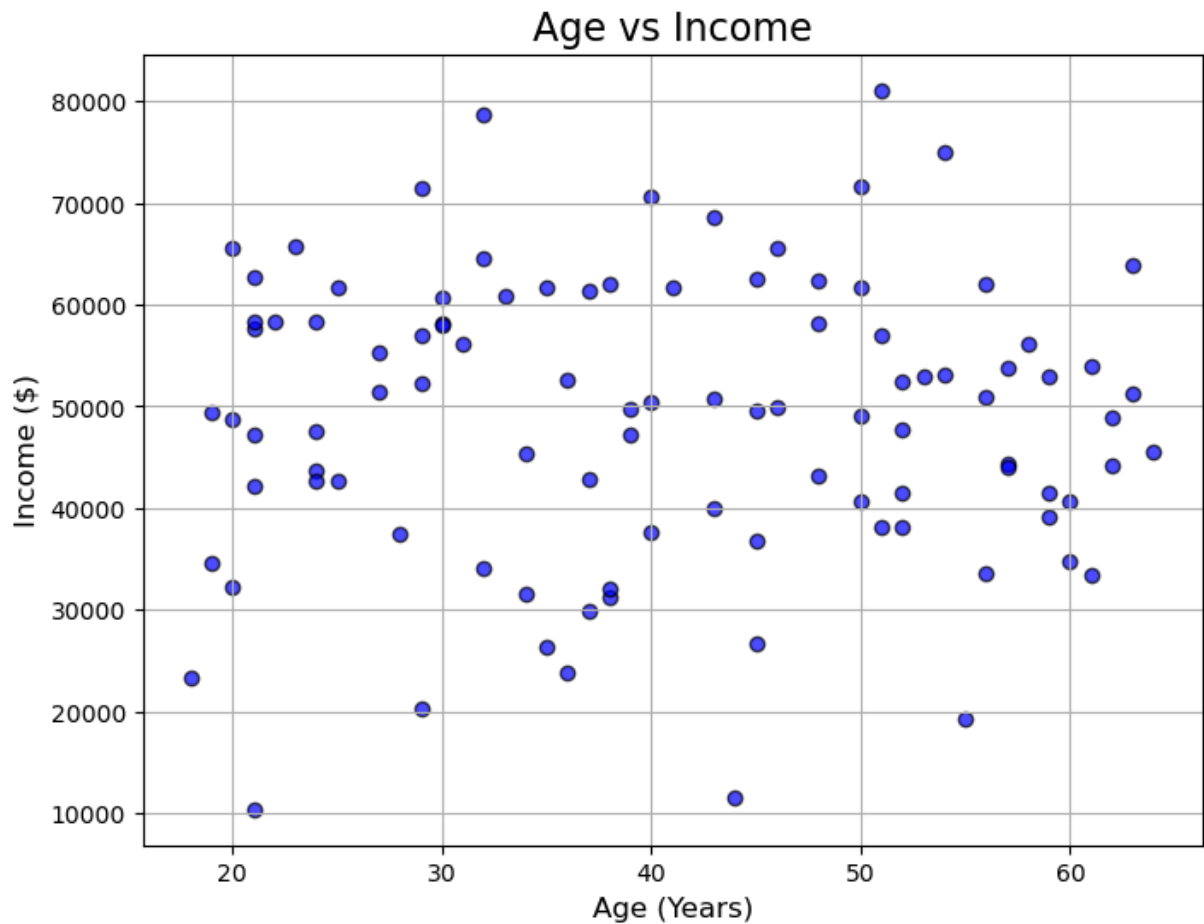
plt.scatter(
    data["Age"],
    data["Income"],
    color="blue",
    alpha=0.7, #Transparency
    edgecolor="black"
)

plt.title("Age vs Income", fontsize=16)
plt.xlabel("Age (Years)", fontsize=12)
plt.ylabel("Income ($)", fontsize=12)

plt.grid(True)

plt.show()

```



Example 2: Customized Line Plot

```
In [5]: months = np.arange(1,13)

sales = [15,18,21,25,28,35,40,42,39,36,30,25]

plt.figure(figsize=(9,5))

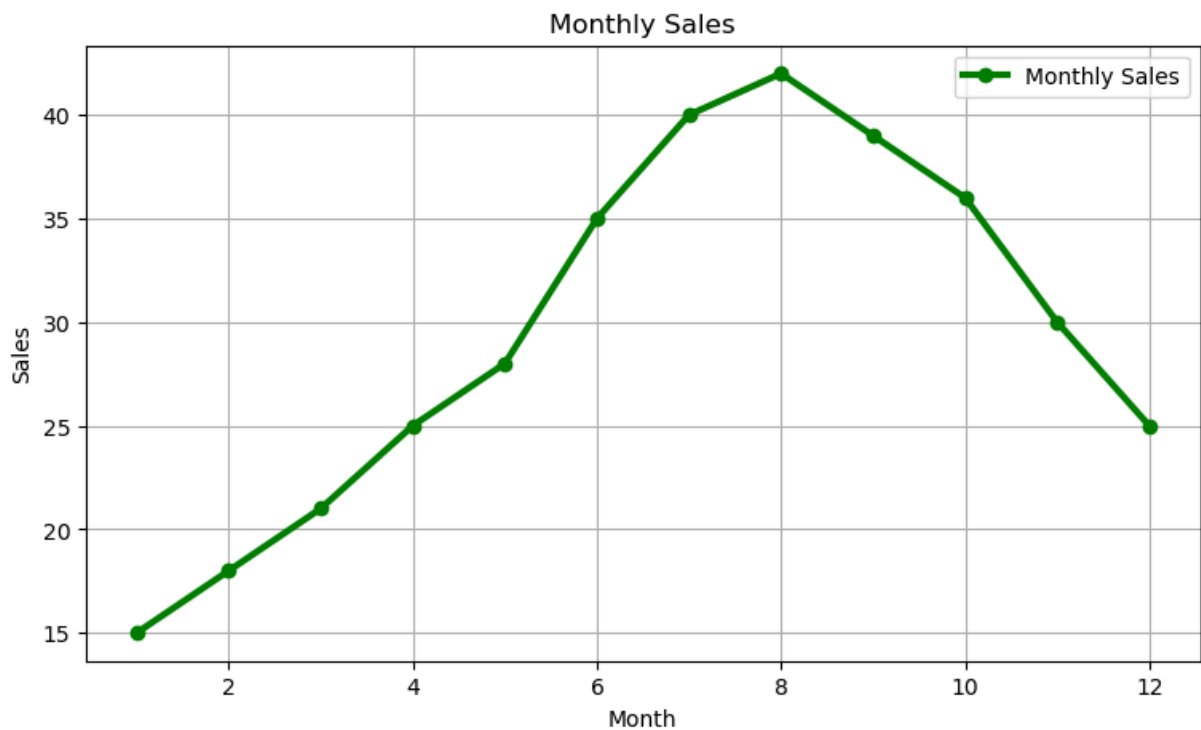
plt.plot(
    months,
    sales,
    marker='o',
    linewidth=3,
    color='green',
    label='Monthly Sales'
)

plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")

plt.legend()

plt.grid(True)

plt.show()
```



Multi-panel Plots (Subplots)

- Subplots allow multiple visualizations within one figure.
- Produces four plots simultaneously.

```
In [6]: fig, axes = plt.subplots(
        nrows=2,
        ncols=2,
        figsize=(12,8)
    )

    axes[0,0].hist(data["Age"])
    axes[0,0].set_title("Age")

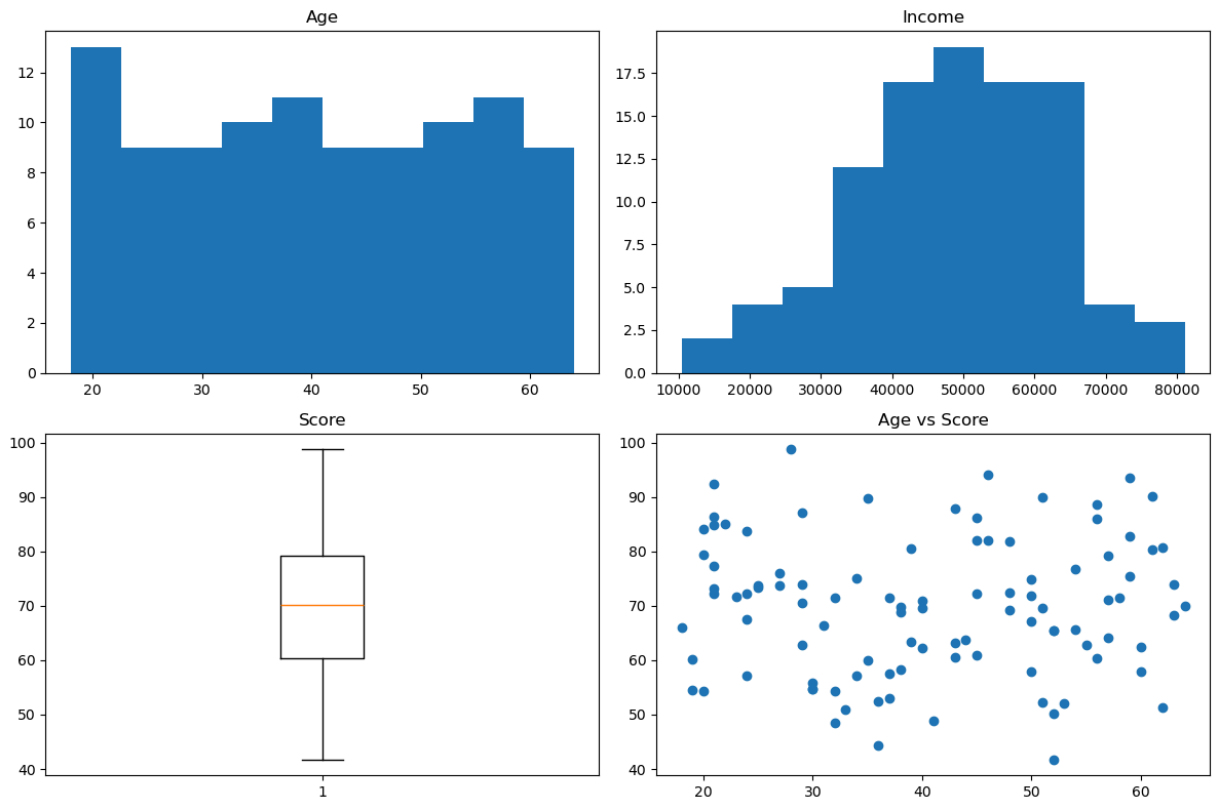
    axes[0,1].hist(data["Income"])
    axes[0,1].set_title("Income")

    axes[1,0].boxplot(data["Score"])
    axes[1,0].set_title("Score")

    axes[1,1].scatter(data["Age"],data["Score"])
    axes[1,1].set_title("Age vs Score")

    plt.tight_layout()

    plt.show()
```



GridSpec Layout

- GridSpec creates flexible figure layouts.
- GridSpec is useful for dashboards.

```
In [7]: from matplotlib.gridspec import GridSpec

fig = plt.figure(figsize=(12,6))

gs = GridSpec(2,3)

ax1 = fig.add_subplot(gs[:,0])
ax2 = fig.add_subplot(gs[0,1:])
ax3 = fig.add_subplot(gs[1,1])
ax4 = fig.add_subplot(gs[1,2])

ax1.hist(data["Age"])
ax1.set_title("Age")

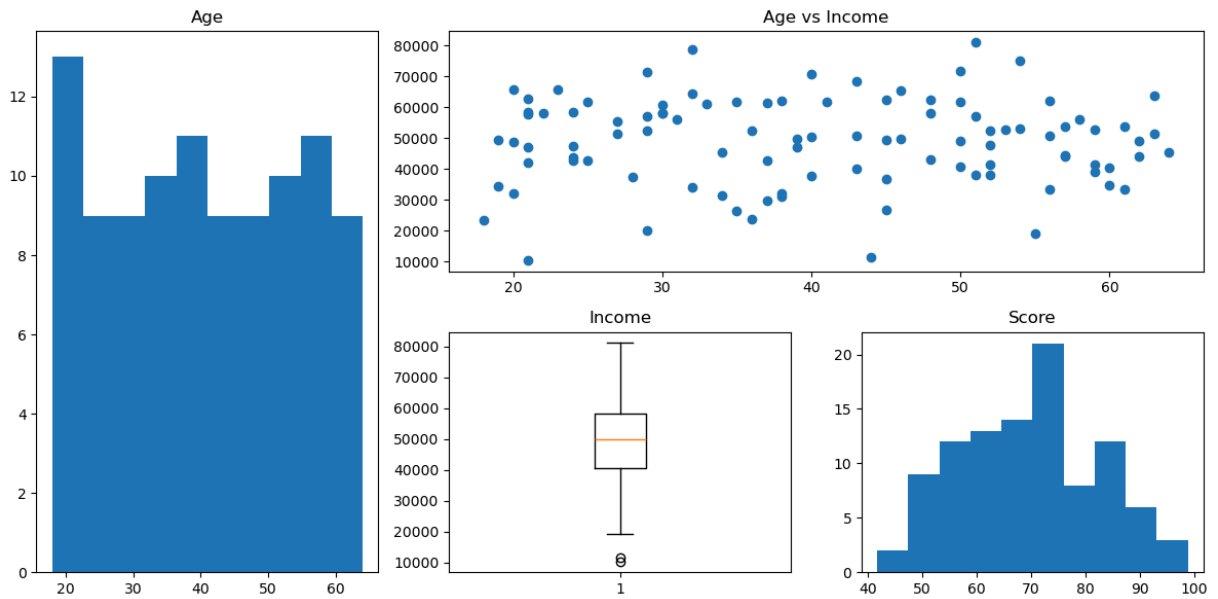
ax2.scatter(data["Age"],data["Income"])
ax2.set_title("Age vs Income")

ax3.boxplot(data["Income"])
ax3.set_title("Income")

ax4.hist(data["Score"])
ax4.set_title("Score")

plt.tight_layout()
```

```
plt.show()
```

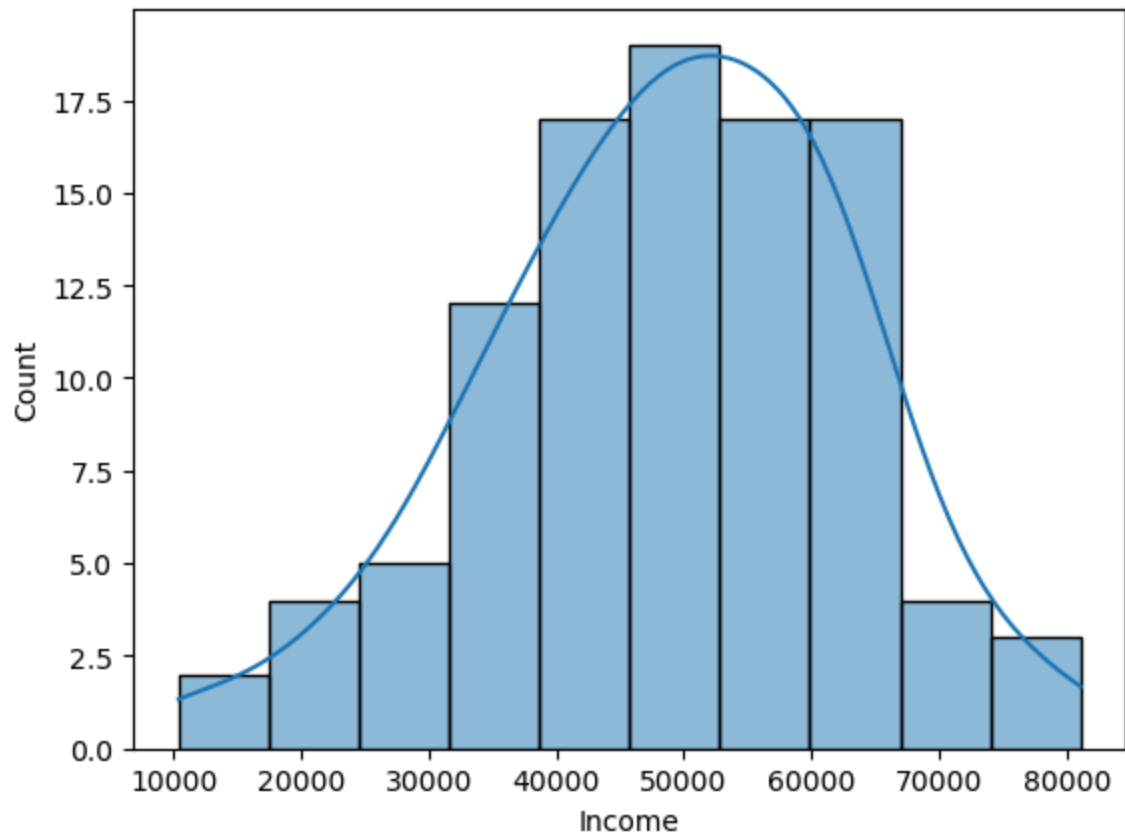


Statistical Graphics with Seaborn

Seaborn builds on Matplotlib and provides attractive statistical graphics.

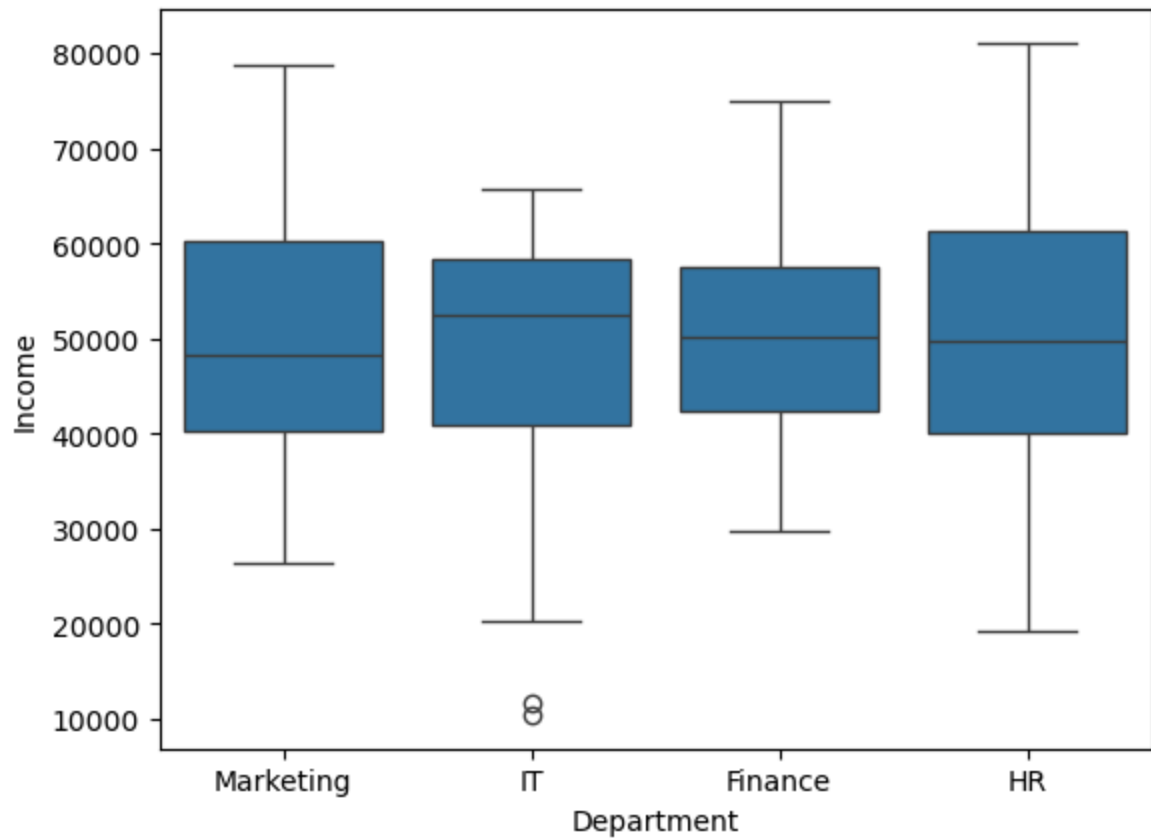
Histogram

```
In [8]: sns.histplot(  
    data=data,  
    x="Income",  
    kde=True  
)  
  
plt.show()
```



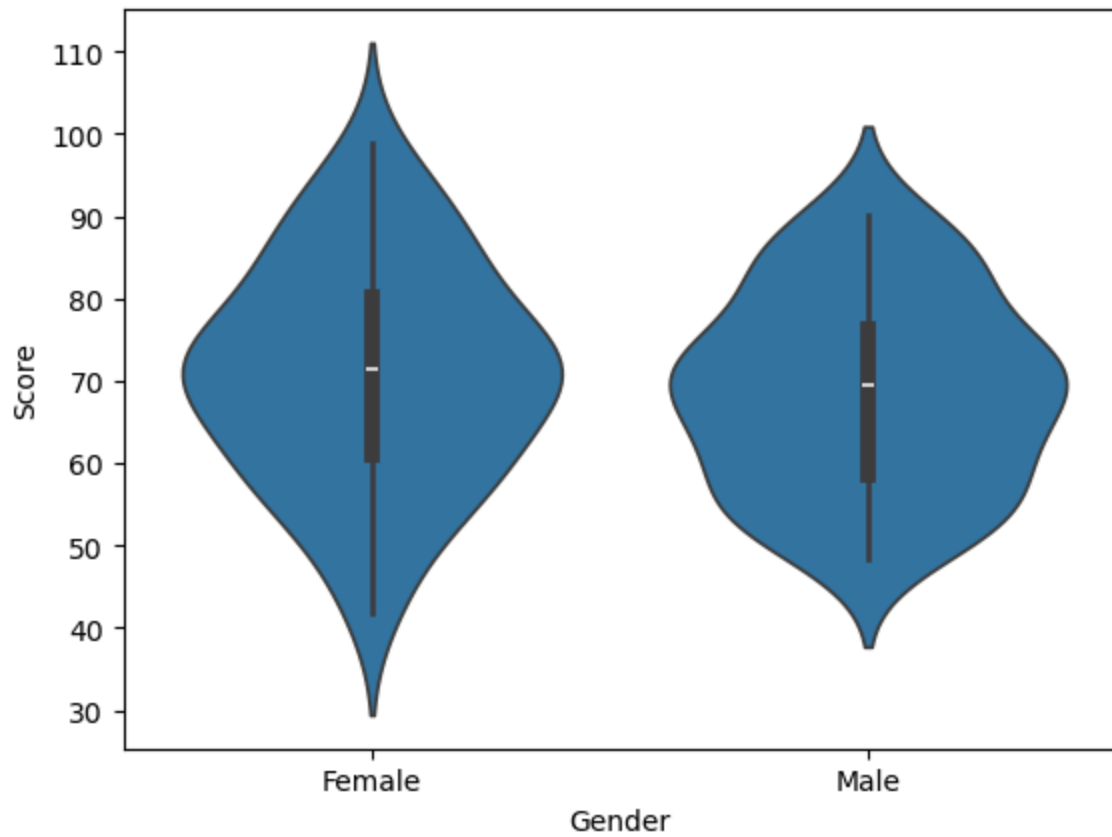
Boxplot

```
In [9]: sns.boxplot(  
    data=data,  
    x="Department",  
    y="Income"  
)  
  
plt.show()
```



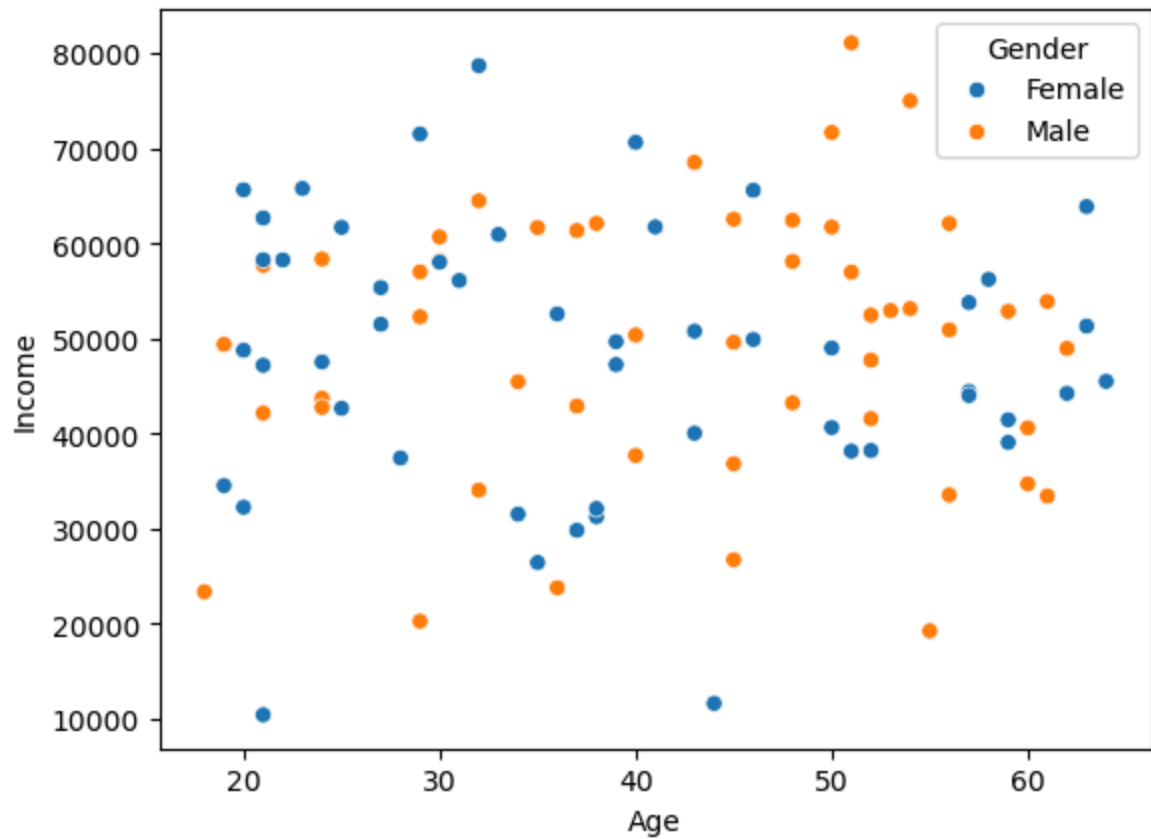
Violin Plot

```
In [10]: sns.violinplot(  
    data=data,  
    x="Gender",  
    y="Score"  
)  
  
plt.show()
```

Scatter Plot

```
In [11]: sns.scatterplot(  
    data=data,  
    x="Age",  
    y="Income",  
    hue="Gender"  
)  
  
plt.show()
```

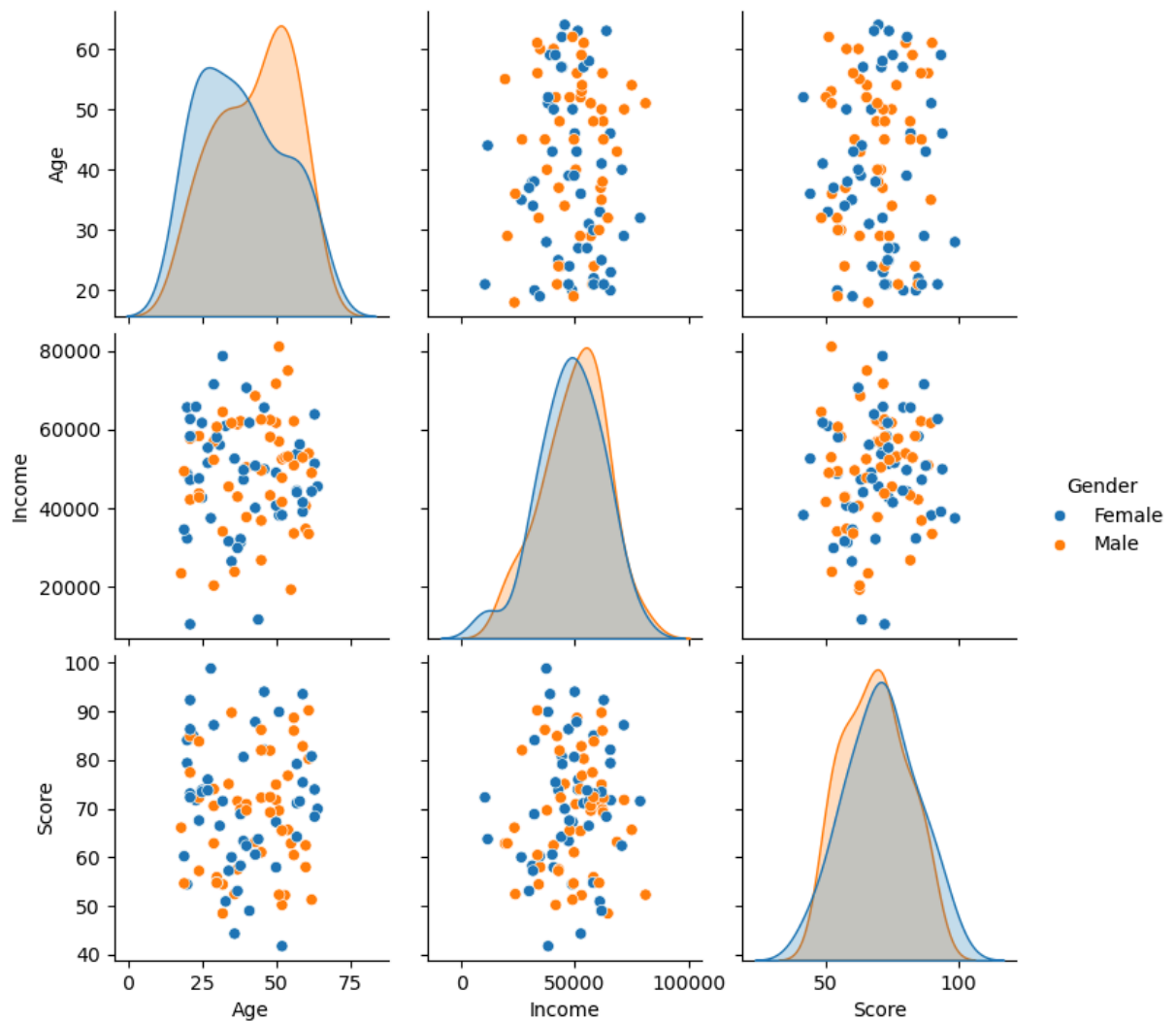


Pair Plot

Useful for Exploratory Data Analysis.

```
In [12]: sns.pairplot(  
    data,  
    hue="Gender"  
)
```

```
Out[12]: <seaborn.axisgrid.PairGrid at 0x211a52c7e00>
```



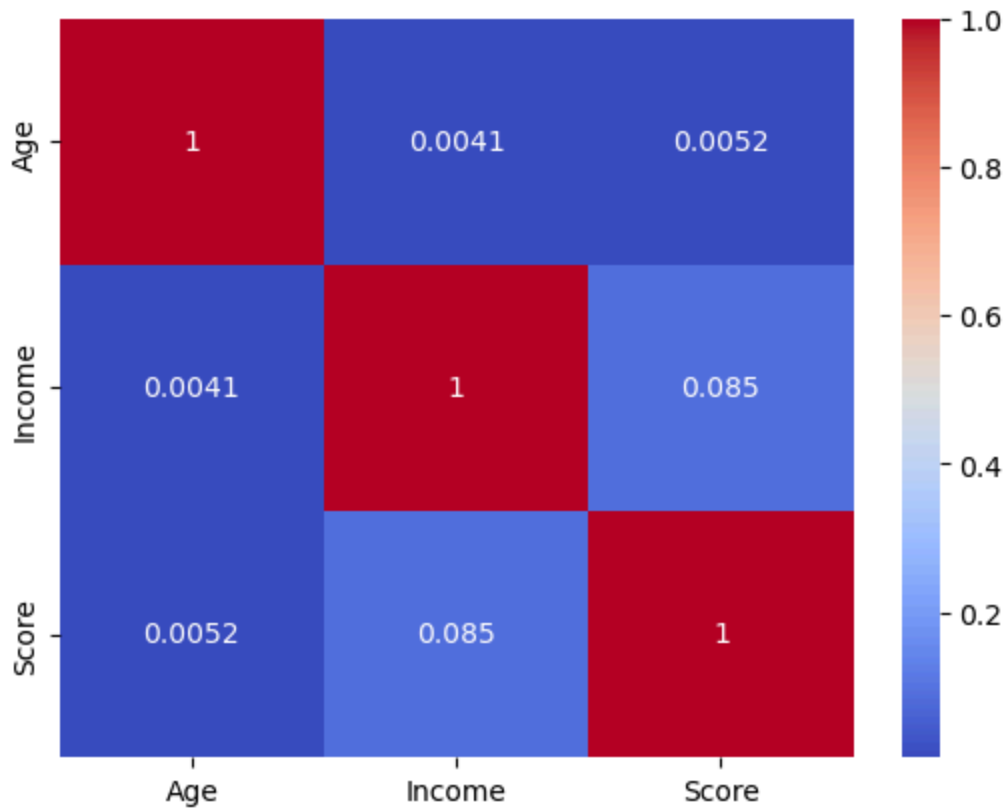
Heatmap

Correlation Matrix

```
In [13]: corr = data.select_dtypes(include="number").corr()

sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm"
)

plt.show()
```



Interactive Visualizations using Plotly

Plotly creates interactive charts.

Interactive Scatter Plot

Learner can;

- Zoom
- Pan
- Hover
- Download images

```
In [14]: fig = px.scatter(
    data,
    x="Age",
    y="Income",
    color="Gender",
    hover_data=["Department"]
)

fig.show()
```

Interactive Bar Chart

```
In [15]: dept = data.groupby("Department")["Income"].mean().reset_index()

fig = px.bar(
    dept,
    x="Department",
    y="Income",
    color="Department",
    title="Average Income by Department"
)

fig.show()
```

Interactive Histogram

```
In [16]: fig = px.histogram(
    data,
    x="Income",
    color="Gender"
)

fig.show()
```

Interactive Visualization using Bokeh

Bokeh supports

- Zoom
- Hover
- Pan
- Save
- Linked brushing

```
In [21]: from bokeh.plotting import figure, show
from bokeh.io import output_notebook

output_notebook()

p = figure(
    title="Age vs Income",
    width=700,
    height=400
)

p.scatter(
    "Age",
    "Income",
    source=data,
    size=8,
    marker="circle"
)
```

```
show(p)
```



Loading BokehJS ...

Geospatial Visualization using GeoPandas

Example using the Natural Earth dataset.

```
In [24]: #!/pip install geodatasets
import geopandas as gpd
import geodatasets

path = geodatasets.get_path("naturalearth.land")
world = gpd.read_file(path)

world.head()
```

```
Out[24]:
```

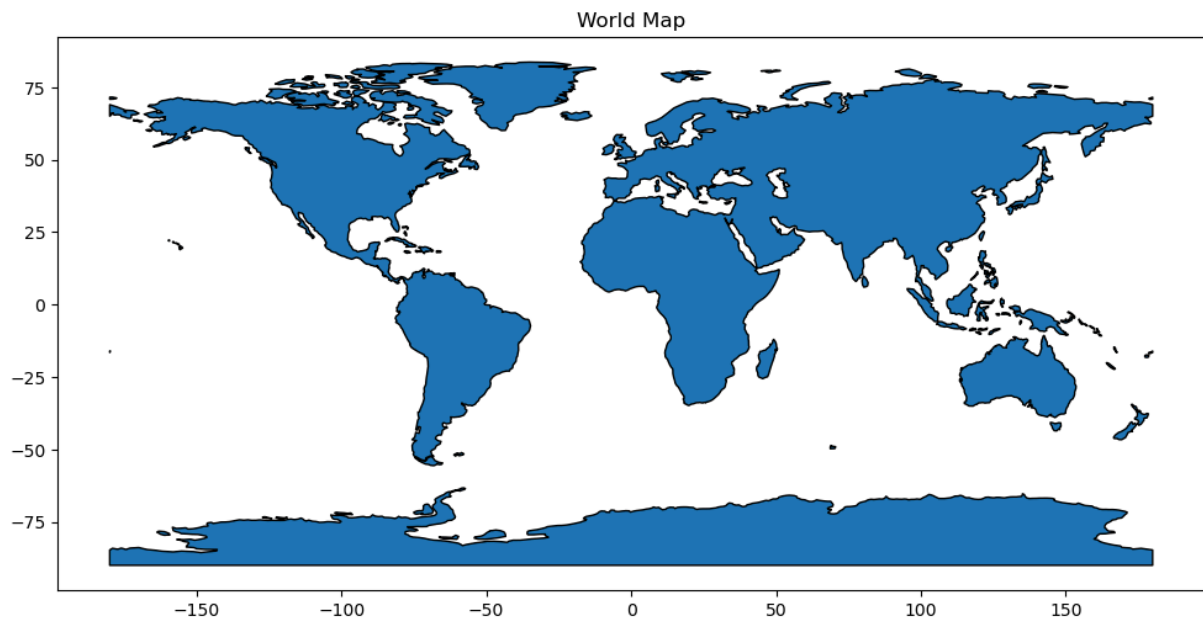
	featurecla	scalerank	min_zoom	geometry
0	Land	1	1.0	POLYGON ((-59.57209 -80.04018, -59.86585 -80.5...
1	Land	1	1.0	POLYGON ((-159.20818 -79.49706, -161.1276 -79....
2	Land	1	0.0	POLYGON ((-45.15476 -78.04707, -43.92083 -78.4...
3	Land	1	1.0	POLYGON ((-121.21151 -73.50099, -119.91885 -73...
4	Land	1	1.0	POLYGON ((-125.55957 -73.48135, -124.03188 -73...

Plot World Map

```
In [25]: world.plot(
    figsize=(12,8),
    edgecolor='black'
)

plt.title("World Map")

plt.show()
```



GitHub GeoJSON

```
In [26]: import geopandas as gpd

url = "https://raw.githubusercontent.com/johan/world.geo.json/master/countries.geo."

world1 = gpd.read_file(url)

world1.head()
```

```
Out[26]:
```

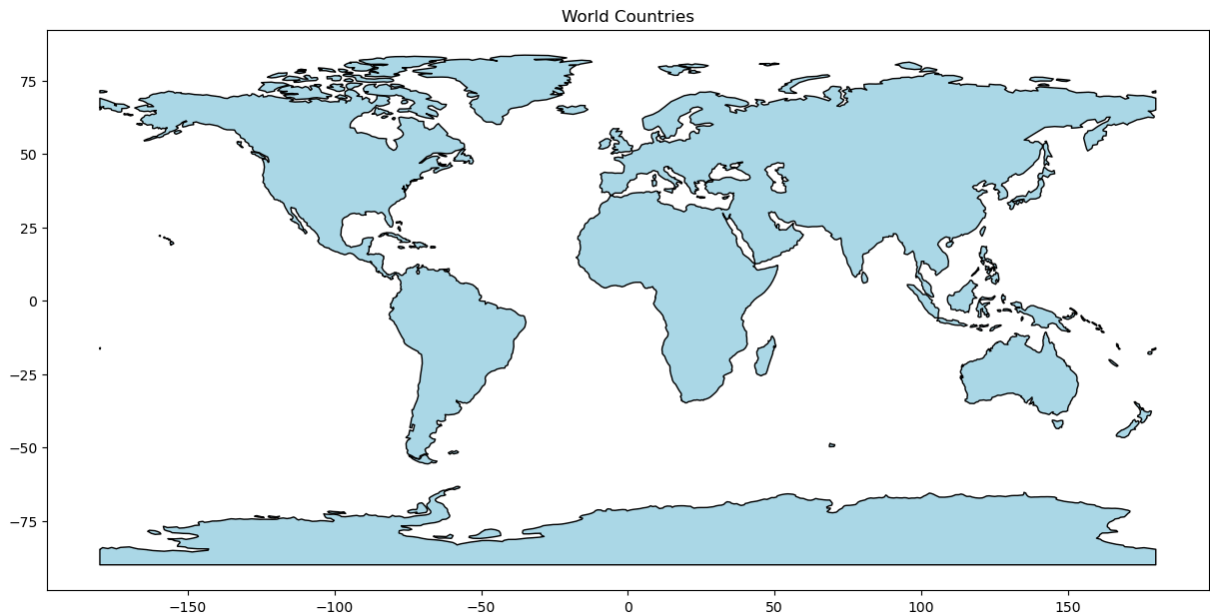
	id	name	geometry
0	AFG	Afghanistan	POLYGON ((61.21082 35.65007, 62.23065 35.27066...
1	AGO	Angola	MULTIPOLYGON (((16.32653 -5.87747, 16.57318 -6...
2	ALB	Albania	POLYGON ((20.59025 41.8554, 20.46318 41.51509,...
3	ARE	United Arab Emirates	POLYGON ((51.57952 24.2455, 51.75744 24.29407,...
4	ARG	Argentina	MULTIPOLYGON (((-65.5 -55.2, -66.45 -55.25, -6...

Plot the Map

```
In [27]: import matplotlib.pyplot as plt

world.plot(
    figsize=(15,8),
    edgecolor="black",
    color="lightblue"
)

plt.title("World Countries")
plt.show()
```



Best for Country Maps

This is the preferred approach for GeoPandas 1.x.

```
In [28]: import geopandas as gpd

url = "https://naturalearth.s3.amazonaws.com/110m_cultural/ne_110m_admin_0_countries"

world = gpd.read_file(url)

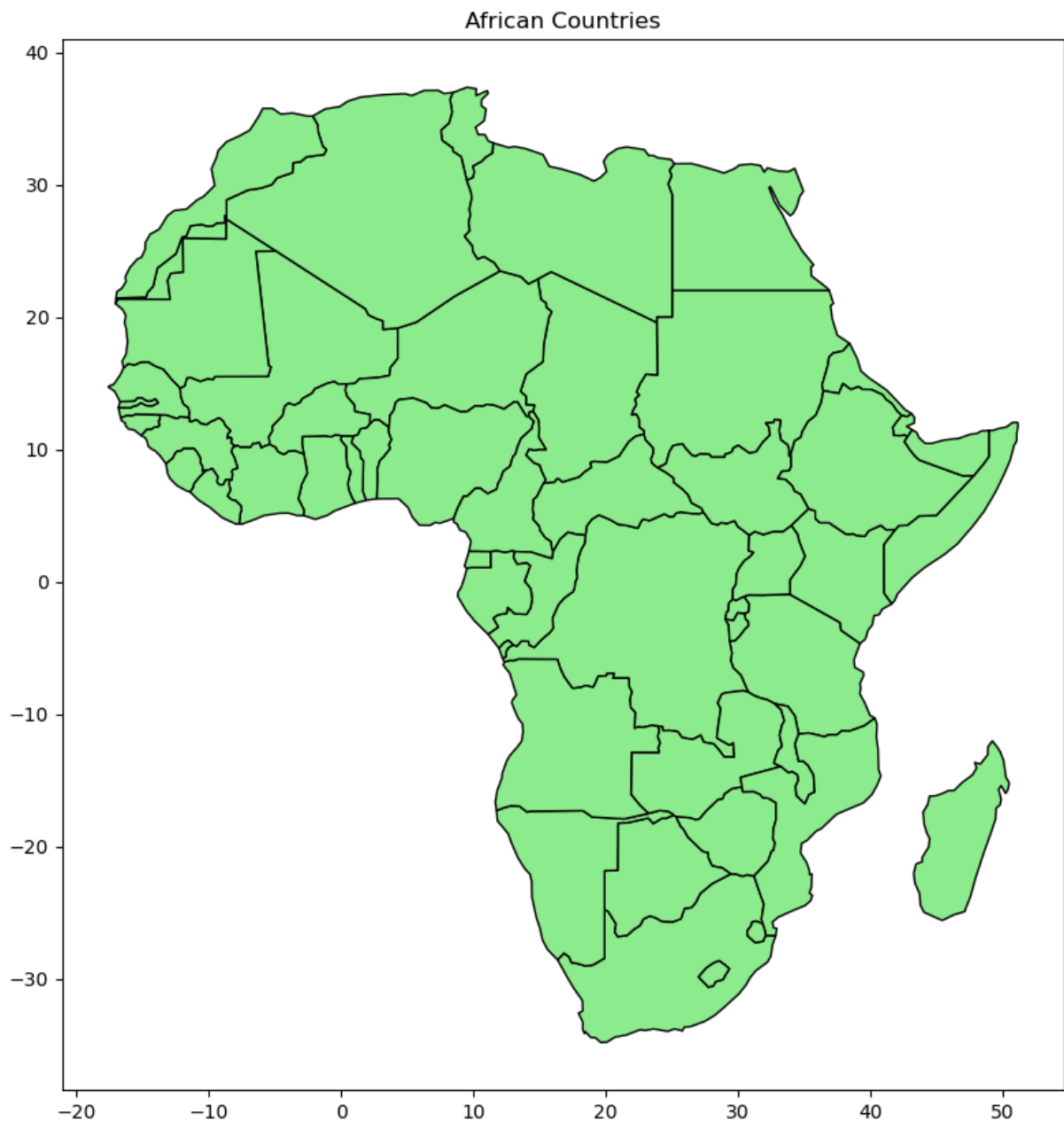
# print(world.columns)
```

Highlight African Countries

```
In [29]: africa = world[world["CONTINENT"] == "Africa"]

africa.plot(
    figsize=(10,10),
    color="lightgreen",
    edgecolor="black"
)

import matplotlib.pyplot as plt
plt.title("African Countries")
plt.show()
```

Interactive Maps using Folium

Create a map centered on Nairobi.

```
In [30]: import folium

m = folium.Map(
    location=[-1.286389, 36.817223], # Nairobi
    zoom_start=6
)
m
# Create a map centered on Nairobi.
```

Out[30]:

Add Marker

Add Circle

```
In [31]: folium.Marker(  
    location=[-1.286389, 36.817223],  
    popup="Nairobi",  
    tooltip="Click me"  
).add_to(m)  
m
```

Out[31]:

Visualizing Multiple Locations

```
In [32]: locations = pd.DataFrame({
    "City":["Nairobi","Mombasa","Kisumu"],
    "Latitude":[-1.286, -4.043, -0.102],
    "Longitude":[36.817,39.668,34.761]
})

m = folium.Map(location=[-1.5,37],zoom_start=6)

for _, row in locations.iterrows():
    folium.Marker(
        [row["Latitude"],row["Longitude"]],
        popup=row["City"]
    ).add_to(m)

m
```

Out[32]:

Seaborn Themes

```
In [33]: sns.set_theme(style="whitegrid")
sns.set_theme(style="darkgrid")

sns.set_theme(style="white")

sns.set_theme(style="ticks")
```

Practice Exercises

1. Create a customized scatter plot of Age vs Score with a title, labels, legend, and grid.
2. Build a 2×3 subplot containing a histogram, boxplot, scatter plot, line plot, bar chart, and heatmap.
3. Use GridSpec to create a dashboard where one plot spans two rows.
4. Create a Seaborn pairplot coloured by department.
5. Develop an interactive Plotly dashboard showing income distributions across departments.
6. Create a Folium map displaying the locations of Kenya's major cities with markers and pop-up information.
7. Download county-level malaria data for Kenya and produce a choropleth map using GeoPandas.